

PORT 0

José Miguel Alves Melo dos Santos

Dissertação para obtenção do Grau de Mestre em
Design e Desenvolvimento de Jogos Digitais
(2º ciclo de estudos)

Orientador: Doutor Frutuoso Gomes Mendes da Silva

Covilhã, Junho 2026

Declaração de Integridade

Eu, NOME DO Aluno, que abaixo assino, estudante com o número de inscrição Mxxxxx de Design e Desenvolvimento de Jogos Digitais da Faculdade de Artes e Letras, declaro ter desenvolvido o presente trabalho e elaborado o presente texto em total consonância com o **Código de Integridades da Universidade da Beira Interior**.

Mais concretamente afirmo não ter incorrido em qualquer das variedades de Fraude Académica, e que aqui declaro conhecer, que em particular atendi à exigida referência de frases, extratos, imagens e outras formas de trabalho intelectual, e assumindo assim na íntegra as responsabilidades da autoria.

Universidade da Beira Interior, Covilhã 04/06/2026

Assinatura

Agradecimentos

Para começar, agradeço ...

Agradeço a todos pela ajuda ao longo desta jornada.

Que o futuro reserve mais.

Trabalho desenvolvido no "Instituto de Telecomunicações" sob orientação do Prof. Frutuoso Silva.

Resumo

....

Palavras-chave

Jogo Educacional, Jogos Sérios, Jogo de Matemática, Unity

Abstract

This work

Keywords:

Educational Games, Serious Games, Math Games, Unity

Conteúdo

Lista de Figuras

Acrónimos

Capítulo 1

Introdução

1.1 Introdução

O presente trabalho surge no âmbito do Mestrado em Design e Desenvolvimento de Jogos Digitais da **UBI!** (**UBI!**) e foca-se na conceção e implementação de um videojogo de simulação e gestão de infraestruturas tecnológicas. O projeto propõe uma experiência imersiva onde o jogador assume o papel de gestor de um pequeno centro de dados (data center), tendo como objetivo primordial a viabilidade económica da operação por uma gestão estratégica de recursos. A abordagem adotada procura equilibrar o rigor técnico de conceitos reais de computação com o entretenimento, utilizando métricas e componentes de hardware autênticos para reforçar a imersão do utilizador.

A estrutura deste relatório de projeto reflete o percurso de desenvolvimento do jogo, começando por contextualizar a relevância do tema e as motivações que levaram à sua escolha. Ao longo dos capítulos subsequentes, é apresentado o estado da arte, onde se analisam títulos de referência no mercado e literatura científica relevante, seguido do detalhe das decisões de design e das técnicas de implementação utilizadas. Finalmente, o documento descreve os processos de teste e avaliação que validam o protótipo desenvolvido, culminando numa reflexão sobre as contribuições do trabalho e as perspetivas para desenvolvimentos futuros na área dos jogos educativos e de simulação.

1.2 Motivação

A motivação para o desenvolvimento deste projeto surge da intersecção entre a complexidade da gestão de infraestruturas tecnológicas e o potencial lúdico dos simuladores modernos. Embora o tema de gestão de data centers possa parecer, à primeira vista, estritamente técnico ou árido, existe um ecossistema rico em mecânicas de estratégia, logística e resolução de problemas que se traduzem naturalmente em experiências de jogo envolventes.

Um dos principais motores deste trabalho é a vontade de desmistificar conceitos de computação de alto desempenho, como o consumo energético em Watts ou a capacidade de processamento em TFLOPS, integrando-os num ciclo de jogo onde o conhecimento é uma ferramenta para o sucesso e não uma barreira à entrada. A utilização de unidades e métricas reais visa reforçar a imersão e conferir um peso estratégico às decisões do jogador, permitindo uma aprendizagem orgânica através da simulação.

Contudo, a motivação central não reside apenas na simulação fiel, mas sim na criação de um produto onde a diversão é o fator predominante. Ao contrário de simuladores puramente técnicos, este projeto adota uma estética inspirada na ficção e elementos de fantasia para garantir um ambiente visualmente apelativo e menos formal. A inclusão de sistemas de combate e mini-jogos baseados em terminais virtuais serve o propósito vital de quebrar a monotonia da gestão económica, introduzindo dinamismo e momentos de adrenalina que mantêm o jogador num estado de flow constante.

Em suma, este projeto é motivado pela convicção de que é possível construir um jogo "sério" que não se leve demasiado a sério. O objetivo é proporcionar uma experiência recompensadora onde o jogador se sinta instigado a otimizar o seu centro de dados, não por obrigação académica, mas pelo puro entretenimento de superar desafios, gerir recursos e proteger a sua infraestrutura num universo digital vibrante.

1.3 Organização do documento

Este documento é organizado segundo os seguintes capítulos:

1. Introdução - ...

2. Estado da Arte - ...

3. Design do Jogo - ...

4. Implementação do Jogo - ...

5. Teste e Avaliação do Jogo - ...

6. Conclusão e Trabalho Futuro - ...

Capítulo 2

Estado da Arte

2.1 Introdução

O presente capítulo tem como finalidade apresentar o estado da arte relativo à temática em análise, reunindo e examinando criticamente as principais contribuições científicas e jogos desenvolvidas até ao momento. A compreensão do panorama atual do conhecimento revela-se fundamental para enquadrar o projeto.

Neste capítulo serão, assim, analisados um conjunto de jogos e artigos, procurando estabelecer um enquadramento crítico que permita compreender a evolução do conhecimento na área e sustentar o desenvolvimento apresentado nos capítulos subsequentes.

2.2 Vídeo Jogos

2.2.1 Data Center Demo

Waseku (2026) está a desenvolver o jogo Data Center (ver Figura ??), que atualmente está em versão Demo na Steam e com previsão de lançamento para o dia 31 de março de 2026, é um jogo do estilo simulador técnico que foca na representação tridimensional e na gestão de infraestruturas de TI. Ao contrário de jogos puramente recreativos, a sua prioridade é a fidelidade técnica e a visualização de métricas em tempo real. Alguns dos pontos mais explorados por ele são:



Figura 2.1: Data Center Demo – Capa do jogo.

- Gestão de Infraestrutura Física: Apresenta para o jogador a disposição das *racks*, unidades de distribuição de energia (PDUs) e sistemas de arrefecimento (CRACs). Demonstrando como o layout físico impacta diretamente a eficiência operacional (ver Figura ??). O jogador terá de fazer a instalação de uma infraestrutura passando pela várias etapas, como montagem, conexão de cabos e configuração de equipamentos.



Figura 2.2: Data Center Demo – Configuração de racks, ligação de cabos.

- Visualização Térmica: Uma das funcionalidades mais marcantes é a simulação de fluxo de ar e calor. O "jogo" mostra visualmente o impacto de corredores quentes e frios, permitindo que o jogador identifique *hotspots* que poderiam levar à falha do *hardware*.
- Simulação de Carga e Falhas: Permite ao jogador simular picos de tráfego ou falhas em componentes específicos (como a queda de um gerador ou UPS) para observar como o sistema de redundância reage.
- Métricas de Sustentabilidade: O *software* calcula e apresenta indicadores de desempenho como o PUE (Power Usage Effectiveness).

O projeto desenvolvido apresenta diversas semelhanças com o Data Center Demo, nomeadamente ao nível do contexto de jogo, da tipologia de objetos representados e do cenário. Contudo, essas características são retratadas de forma mais simplificada e incluem alguns elementos de fantasia. A principal intenção é privilegiar a componente lúdica e a experiência de entretenimento, sem, no entanto, excluir por completo os elementos técnicos subjacentes à simulação.

2.2.2 PC Building Simulator

Claudiu Kiss (2019) desenvolveu o PC Building Simulator (ver Figura ??) com a proposta de ser um jogo de montagem de computadores, que permite ao jogador passar pelas várias etapas de montagem de um computador pessoal. O jogo destaca-se pela representação fiel de componentes e pela introdução de conceitos técnicos como compatibilidade de *sockets*.



Figura 2.3: PC Building Simulator – Capa do jogo.

- Instalação de *hardware*, desde da colocação de *hardware* até conexão de cabos;
- Instalação de sistemas operativos;
- Instalação de *softwares* adicionais e realização de teste.

O jogo destaca-se pela representação fiel de componentes (CPU,GPU,RAM,etc...) e pela introdução de conceitos técnicos como compatibilidade de *sockets* (ver Figura ??).



Figura 2.4: PC Building Simulator – Montagem de um computador.

A representação realista do hardware, bem como a simulação da instalação de sistemas operativos, foram também retratadas no projecto desenvolvido, ainda que de forma mais simplificada, lúdica e adaptada ao respectivo contexto. Importa salientar que, à data da redação deste texto, já se encontra disponível o PC Building Simulator 2. No entanto, para efeitos deste trabalho, os aspetos apresentados são comuns a ambas as versões.

2.2.3 Startup Company

Hovgaard Games (2020) desenvolveu o jogo Startup Company (ver Figura ??), que apresenta uma proposta centrada na simulação de gestão empresarial na área da tecnologia. Neste jogo, o jogador assume a responsabilidade de criar e desenvolver uma empresa tecnológica a partir do zero , gerindo recursos materiais e humanos.



Figura 2.5: Startup Company – Capa do jogo.

Compete-lhe gerir de forma estratégica os recursos materiais (ver Figura ??), nomeadamente os espaços e o local de trabalho, bem como os recursos humanos, incluindo programadores, líderes de projeto, designers, profissionais de marketing e responsáveis pela gestão de servidores (ver Figura ??).

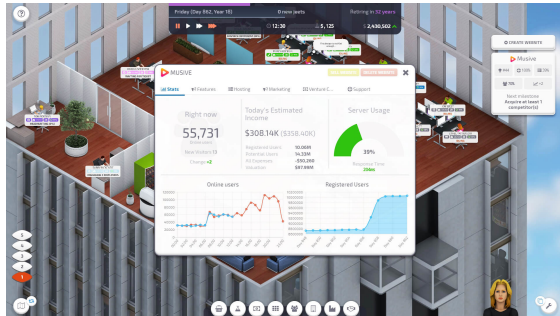


Figura 2.6: Startup Company – Métricas que o jogo disponibiliza para o jogador.

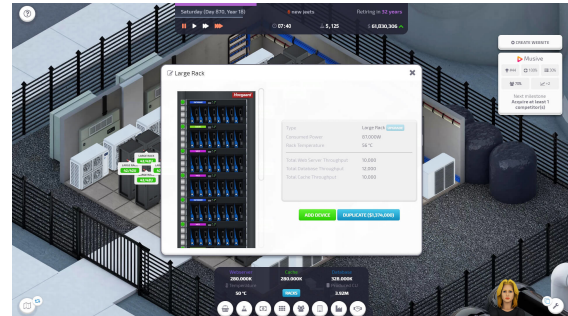


Figura 2.7: Startup Company – Configurações de servidores.

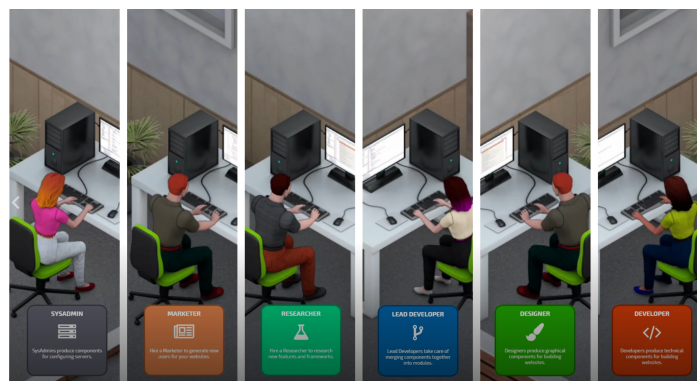


Figura 2.8: Startup Company – Exemplo de recursos humanos disponíveis. Retirado de https://store.steampowered.com/app/606800/Startup_Company/?l=portuguese

Além dessa parte, gestão de cliente, custo, consumos da infraestrutura assim como rendimentos, são fatores relevantes neste jogo (ver Figura ??). Serviram de inspiração para a criação de funcionalidades no projeto, tais como pedido de serviços feito por clientes para o jogador, e métricas de consumo e desempenho.

2.2.4 Grey Hack

Loading Home (2017) está a desenvolver o jogo Grey Hack (ver Figura ??), o qual está em acesso antecipado na plataforma Steam. Estes títulos focam-se na simulação de ambientes de rede e *hacking* por interfaces que mimetizam sistemas operativos reais, utilizando terminais de linha de comando (CLI).



Figura 2.9: Grey Hack - Capa do jogo.

Grey Hack utiliza terminais de linha de comando (CLI) e visualizações de topologia de rede para criar uma experiência imersiva, onde o jogador interage com o sistema de forma lógica e textual (ver Figura ??).

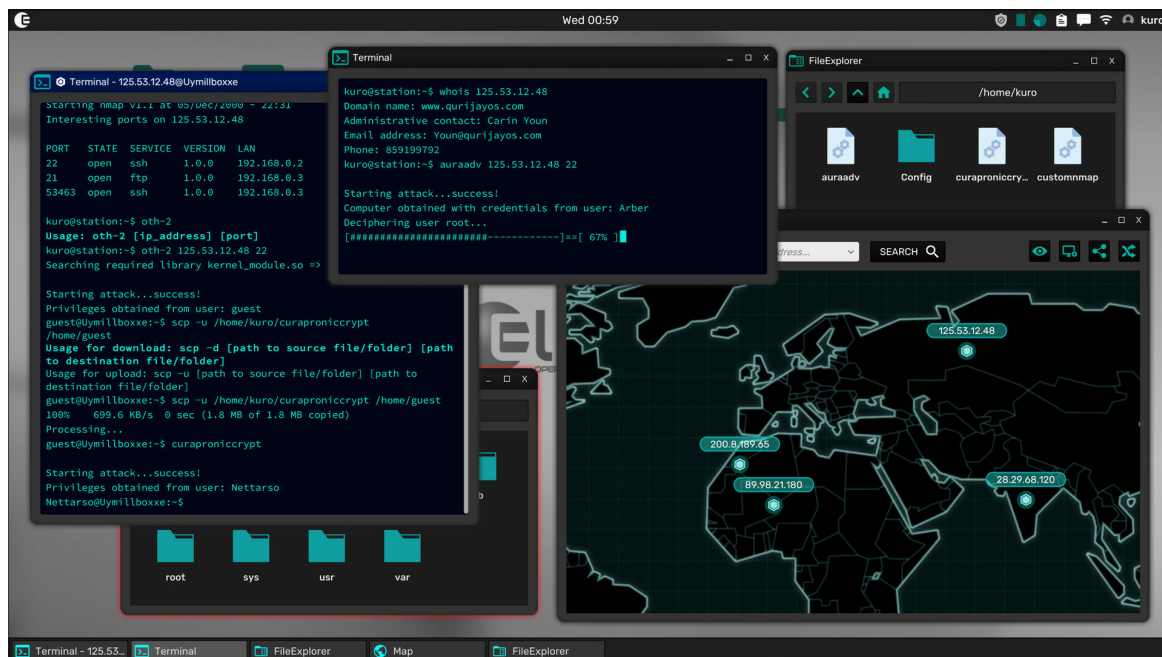


Figura 2.10: Grey Hack - UI de terminal. Retirado de https://store.steampowered.com/app/605230/Grey_Hack

Transpondo para o projeto, estas referências servem de base para a camada de interação (UI/UX). O projeto adota a utilização de terminais virtuais para a conclusão de *mini games*.

2.2.5 Software Inc.

Coredumping (2015) desenvolveu o jogo Software Inc. (ver Figura ??), o qual apresenta um sistema de construção complexo que inclui a gestão de eletricidade, temperatura e disposição de salas. O jogo exige que o jogador planeie a colocação de sistemas de arrefecimento e geradores para garantir a integridade do hardware.



Figura 2.11: Software Inc – Capa do jogo.

A influência de Software Inc. no projeto reside no planeamento de sistemas. Tais como a necessidade de gerir a gestão da rede elétrica (capacidade das UPS e geradores) são mecânicas de logística urbana adaptadas para o ambiente de jogo.

2.3 Artigos científicos

Para além dos videojogos anteriormente referenciados, existe um conjunto de artigos que se enquadram na área científica do projeto e que foram fundamentais para sustentar o seu desenvolvimento, estabelecendo uma ponte importante entre o universo dos jogos e a aplicação real. Um exemplo desta correlação é o trabalho de Hosseini e Vakili (2018), que demonstra como conceitos teóricos complexos, como a Teoria dos Jogos, são aplicados na prática para identificar centros de dados vulneráveis em redes de computação em nuvem através da análise de custos e tempos de resposta. A inclusão desta referência no estado da arte serve para validar como as dinâmicas de gestão e os desafios de infraestrutura abordados no projeto refletem preocupações reais e atuais da computação científica, reforçando a relevância do contexto em que o utilizador é inserido.

A metodologia de design é apoiada por Silva (2019), que descreve diretrizes para a criação de jogos com objetivos educacionais. Embora o projeto desenvolvido se defina especificamente como um adding game, e não como um jogo sério tradicional, a metodologia apresentada foi relevante na fase de conceptualização para estruturar a integração de conteúdos técnicos de forma equilibrada. Esta abordagem é complementada pelos conceitos de macrodesign de Shay e Langbeheim (2022), que identificam

fatores cruciais como a fantasia, o mistério e o desafio para garantir que o rigor do tema não prejudica a imersão lúdica. Para o desenvolvimento deste trabalho, a aplicação destes fatores justifica a inclusão de elementos de fantasia e métricas simplificadas em cima de conhecimentos reais, garantindo que o jogador se mantenha motivado por mecânicas envolventes e permitindo a criação de um conceito de jogo que soma conhecimentos ao jogador mantendo a diversão como fator principal.

Capítulo 3

Técnicas e Decisões de Design

Neste presente capítulo serão explorados os conceitos de design aplicados no projeto desenvolvido assim como as decisões tomadas durante o processo de desenvolvimento. O design do jogo é um elemento crucial para garantir uma experiência envolvente e satisfatória para os jogadores, envolve a criação de mecânicas com técnicas de design, complementa o projeto como um todo.

Serão apresentados métodos científicos e técnicos utilizados para a construção do jogo, bem como as escolhas pessoais e criativas que influenciaram o desenvolvimento do projeto. O objetivo é estabelecer uma compreensão clara do processo de design e das decisões tomadas.

3.1 Técnicas utilizadas

Nas subsecção consequentes serão descritas e apresentadas as técnicas de design utilizadas no desenvolvimento do projeto, incluindo a aplicação de princípios de design e metodologias de desenvolvimento.

3.1.1 *Framework MDA (Mechanics, Dynamics, Aesthetics)*

O *framework* MDA, desenvolvido por Hunicke et al. (2004), é uma abordagem amplamente utilizada no design de jogos concentrados em três componentes principais: Mecânicas, Dinâmicas e Estéticas. As Mecânicas referem-se às regras e sistemas que governam o jogo, as Dinâmicas são as interações inerentes entre as mecânicas e os

jogadores, e as Estéticas são as emoções e experiências que os jogadores vivenciam ao jogar.

Posto isto, o *framework* MDA foi utilizado como guia para a construção do jogo, e o diagrama apresentado na Figura ?? ilustra a relação entre as mecânicas, dinâmicas e estéticas do jogo desenvolvido.

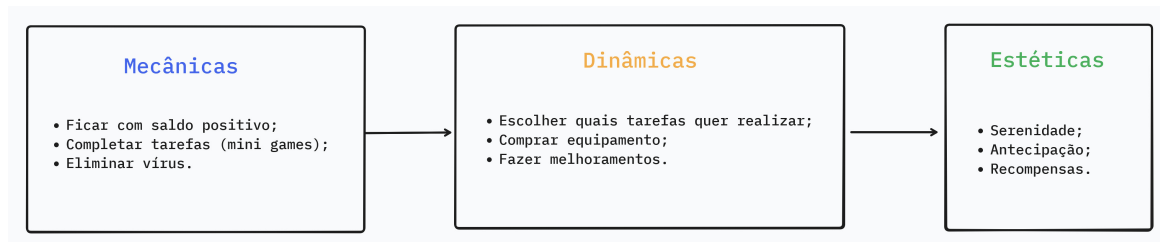


Figura 3.1: Diagrama do *framework* MDA aplicado ao jogo desenvolvido.

3.1.2 Flow

Embora venha da psicologia, o trabalho de Chen (2007) é a base para o design de progressão em jogos. Define o equilíbrio entre a dificuldade do desafio e a habilidade do jogador para evitar o tédio ou a ansiedade. O conceito de *Flow* foi aplicado no design do jogo para garantir que os jogadores se mantenham engajados e motivados a continuar jogando, proporcionando desafios adequados ao nível de habilidade dos jogadores.

Na Figura ?? é possível observar um diagrama que ilustra o conceito de *Flow* aplicado ao design do jogo.

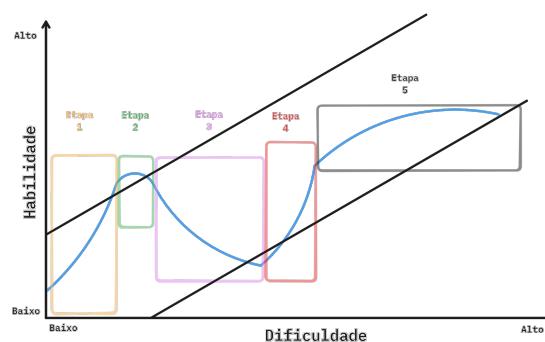


Figura 3.2: Diagrama do conceito de *Flow* aplicado ao design do jogo.

O diagrama é dividido em 5 etapas, onde cada uma delas representam momentos específicos do jogo. A 1ª etapa representa os primeiros momentos do jogo, onde o jogador está a aprender as mecânicas básicas e a perceber o que tem de fazer para progredir, como tal é representada por uma curva crescente.

A 2ª etapa é caracterizada pela adaptação do jogador às mecânicas introduzidas na etapa anterior, como tal a curva de *flow* começa a descer, uma vez que devida a adaptação do jogador, o desafio se torna mais fácil.

A 3ª etapa representa a habituação do jogador às mecânicas, assim como a repetição de desafios semelhantes, o que leva a uma curva de *flow* com um declive negativo, uma vez que o jogador começa a sentir-se entediado e desmotivado a continuar jogando.

A 4ª etapa, por contraste com a anterior, é caracterizada pela introdução de inimigos, o que indiretamente implica a introdução de mecânicas novas e quebra o ciclo até aquele momento, o que leva a um aumento da dificuldade percebida pelo jogador, o que indiretamente justifica o aumento do *flow*.

Por fim a 5ª etapa é caracterizada por uma adaptação, da parte do jogador, às mecânicas introduzidas na etapa anterior, com as mecânicas iniciais, o que resulta numa curva de *flow* com um crescimento bastante moderado, uma vez que o jogador já tem uma base sólida construída com as mecânicas anteriores, o que o torna a *gameplay* muito mais previsível, resultando assim numa curva de *flow* mais estável e menos propensa a declives acentuados.

Em suma, a progressão do jogo utiliza a alternância entre a introdução de novos desafios e períodos de adaptação para modular a experiência do utilizador. Ao gerir estas cinco etapas, o design garante que o jogador no estado de *Flow*, equilibrando a curva de aprendizagem com momentos de novidade, assegurando assim uma experiência envolvente, fluida e recompensadora do início ao fim.

3.1.3 Modelo de *Hook*

No âmbito do game design contemporâneo e do desenvolvimento de produtos digitais, a retenção de utilizadores tornou-se um dos indicadores de desempenho mais críticos. Neste contexto, o Modelo de Hook (*Hook Model*), conceptualizado por Eyal (2014), surge como uma estrutura teórica e prática desenhada para fomentar a criação de hábitos através da interação contínua com o produto.

Ao contrário de abordagens baseadas meramente em publicidade agressiva, o Modelo de Hook foca-se na psicologia do utilizador, ligando os seus problemas e necessidades quotidianas a uma solução tecnológica específica. Este modelo é composto por um ciclo iterativo de quatro fases fundamentais: o Gatilho (*Trigger*), a Ação (*Action*), a Recompensa Variável (*Variable Reward*) e o Investimento (*Investment*).

Posto isto, o Modelo de Hook utilizado no desenvolvimento deste projeto, é caracterizado pelos seguintes elementos:

- *Trigger*: Curiosidade pela temática do jogo, apelo pelo visual e apelo de curiosidade pelo uso de técnicas e tecnologias reais, senso de colecionismo. Retornar ao jogo para obter o dinheiro que está em estado pendente.
- *Ação*: Resolução de puzzles rápidos, cerca de 60 segundos, sobre a forma de *minigames*, otimização de recursos e processos. Derrotar inimigos e fazer melhoramentos gerais.
- *Recompensas*: Obtenção de novos recursos, equipamentos e troféus, (coleccionismo), melhoramento de equipamentos.
- *Investimento*: Investimento de tempo e esforço para a progressão no jogo.

Alguns elementos presente no modelo assim, como troféus e melhoramento de equipamentos (tanto de servidores como de armas), estão presente no conceito de design do jogo, no entanto, não estão contemplados no projeto desenvolvido, uma vez que não eram o foco principal e não existiu tempo suficiente para a sua implementação.

3.2 Decisões de Design

3.2.1 UI! (UI!) e referências técnicas

A *User Interface* (UI!) do jogo está presente durante toda a *gameplay*, como tal construir uma UI! clara e que se enquadre no contexto do jogo, é de suma importância. Essa visão é partilhado por Adams (2014) que argumenta que a UI! deve atuar como uma "extensão do mundo" através do seu estilo visual.

Assim, a UI! do jogo é inspirada no estilo de programas com interfaces de *Command Line Interface* (CLI! (CLI!)), com um design minimalista e funcional, utilizando cores

escuras e elementos visuais que remetem para o contexto de Data Center, como, por exemplo, gráficos de barras para representar o uso de recursos, ou ícones de servidores para representar os equipamentos disponíveis.



Figura 3.3: Exemplo de referência - UI! do Gemini utilizado em CLI!

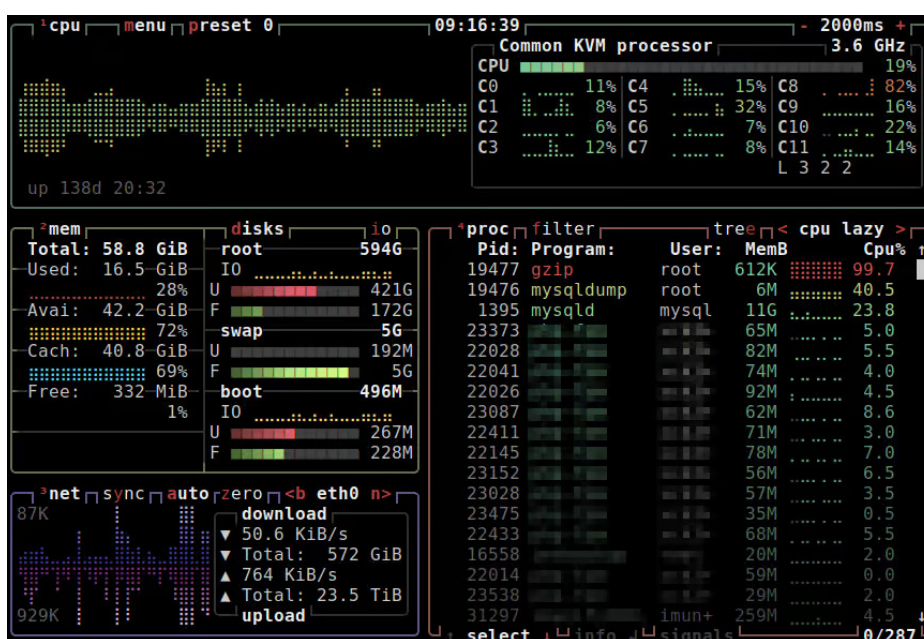


Figura 3.4: Exemplo de referência - UI! do btop

As Figuras ?? e ?? apresentam exemplos de referências utilizadas para a construção da UI! do jogo. A UI! das referências apresentadas é caracterizada pelo uso de cores escuras de fundo, o uso de fonte mono espaçadas, e o uso de gráficos simples, geralmente composto, por barras, pontos, linhas, caracteres ou quadrados.

Esses mesmos elementos podem ser visto da **UI!** da loja, por exemplo, (Figura ??) onde é utilizada uma cor de fundo escura, o uso da fonte JetBrains Mono, que é uma fonte mono espaçada, o uso de texto e o uso de gráficos simples, como os botões e as barras de *status*.

3.2.2 Design de Level

O design de level é o ato de projetar os níveis de um jogo, incluindo a disposição dos elementos, a criação de desafios e a definição do ritmo do jogo. Como tal é um dos pontos mais críticos para a construção de um jogo, uma vez que interfere diretamente na experiência e ritmo percebido pelo jogador.

Posto isto o design de level do jogo foi desenvolvido para ser compacto, mas que, ao mesmo tempo, cada área ofereça uma experiência distinta, que está diretamente relacionada com a curva de *flow* recebido pelo jogador.

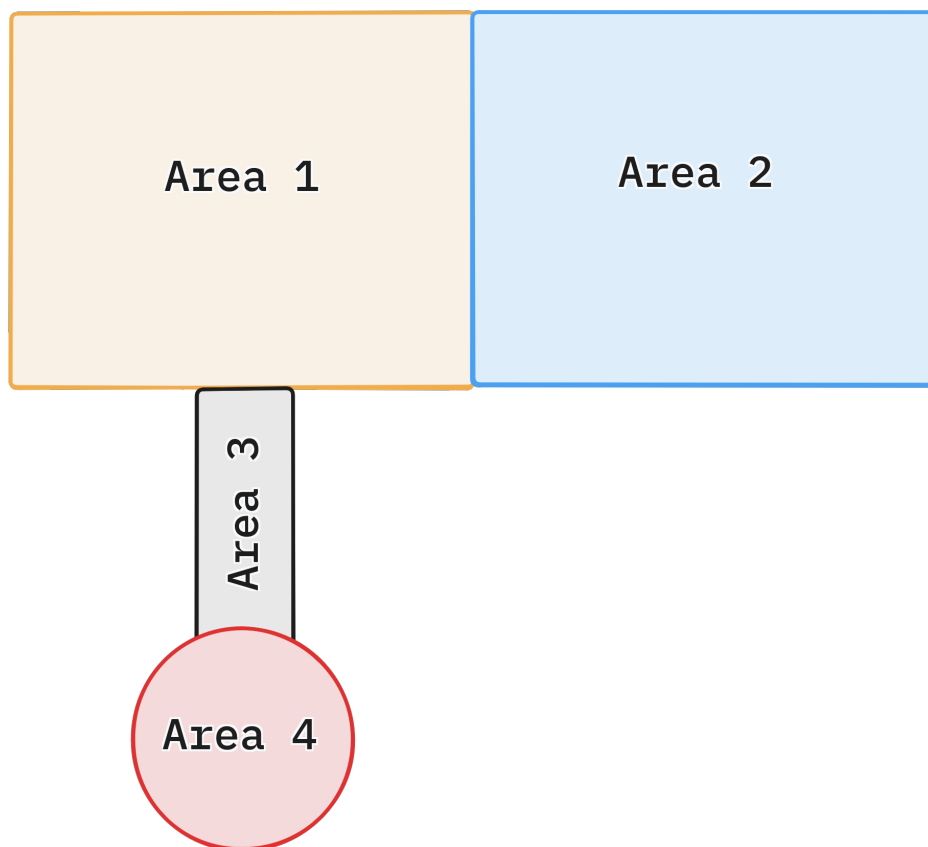


Figura 3.5: Level design do jogo desenvolvido.

Na Figura ?? está representado o design do level do jogo dividido em 4 áreas distintas,

sendo que cada uma tem as seguintes características:

- Área 1: Área principal, onde estão localizados os servidores, e onde o jogador interage com os mesmos para resolver os puzzles, que dão dinheiro como recompensa para o jogador. É nesta área onde o jogador irá passar a maior parte do tempo, e por isso mesmo é que é a área central e a área que serve de união para as outras duas.
- Área 2: Área de conquista e customização, nessa área o jogador poderá ver conquistas obtidas sobre a forma de troféus ou objetos de jogo. Além disso, nesta área o jogador poderá interagir com o cenário para o customizar, como por exemplo interagir com um computador ou aparelho para fazer upgrades no cenário ou customizar cores, etc... Esta área é caracterizada por ser um espaço de relaxamento, sem objetivos nem inimigos.
- Área 3: É um corredor que liga a área 1 à área 4. Não tem nenhum objetivo primário.
- Área 4: Área de combate, nessa área o jogador terá de utilizar de uma metralhadora para derrotar conjuntos de inimigos que irão atacar o data center. Esta área é muito especial pois é nela que existe o rompimento da *gameplay*, e por isso mesmo a necessidade da existência da área 3, uma vez que o corredor representa o distanciamento do conforto proporcionado pelos puzzles da área 1. O formato geométrico dessa área também ajuda a transmitir esse contraste, uma vez que enquanto todas as outras áreas apresentam um formato quadrado, esta área apresenta um formato circular. O formato quadrado geralmente está associado a algo estável, sólido e imóvel, enquanto que o formato circular está associado a algo fluido, móvel e instável. Assim, o formato circular representa a mudança de mentalidade que o jogador tem de ter, pois nessa área o jogador terá de ser rápido e capaz de desenvolver uma *gameplay* dinâmica, que o combate exige. Além disso, a distância geométrica entre a área 4 e a área 2, é a maior de todas, estando em pontas opostas, o que ajuda a transmitir a ideia de algo caótico, uma vez que a área 2 é um espaço de relaxamento.

Todavia no projeto apresentado, a área 2 não foi implementada na sua totalidade, devido à sua complexidade, sobre tudo pela parte de desenvolvimento de troféus e pela parte de customização de cenário.

3.2.3 Notificações

Durante o desenvolvimento do jogo surgir a necessidade de desenvolver um sistema capaz de transmitir *feedbacks* informativos para o jogador. Como, por exemplo, notificar o jogador quando é realizada uma compra, completada uma tarefa, destruído um servidor, etc ...

Segundo Yu et al. (2019), o uso de notificações em jogos, é recomendável e apresenta um efeito positivo no desempenho do jogador.

Para suprir essa necessidade foi desenvolvido um simples sistema de notificações. O sistema envia uma notificação, sobre a forma de texto, ao jogador, sempre que acontece, alguma das ações descritas acima.



Figura 3.6: Notificação – Destruir todos os servidores.



Figura 3.7: Notificação – Compra de um novo servidor.

As Figuras ?? e ?? exemplificação como as notificações são executadas durante o ciclo de *gameplay*.

3.2.4 Referências para a criação de elementos de Jogo e 3D

No processo de criação de elementos de jogo, sobretudo elementos 3D de cenário, houve o cuidado de utilizar referências reais, como por exemplo, fotos de data centers reais, para a construção dos elementos do jogo, no sentido de criar uma experiência mais imersiva para o jogador, além de possibilitar a criação de conexões entre o jogo e o mundo real, o que se torna benéfico para a construção de uma experiência mais lúdica além de facilitar conexões entre o jogo e jogadores que tenham interesse na temática do jogo, ou que tenham experiência prévia.

No entanto, houve uma adaptação dos mesmos, e uma adição de elementos de fantasia, para criar um ambiente mais divertido e envolvente para o jogador, além de proporcionar uma experiência mais dinâmica e variada.

Neste sentido, houve uma pesquisa prévia sobre a estrutura física de um data center, o que resultou num conjunto de elementos de cenário, que são bastante específicos, destes ambientes. Tais como:

- Pavimento técnico;
- Racks de servidores;
- Cabos de rede;
- Sistemas de refrigeração;
- Geradores de energia;
- Equipamentos de rede;

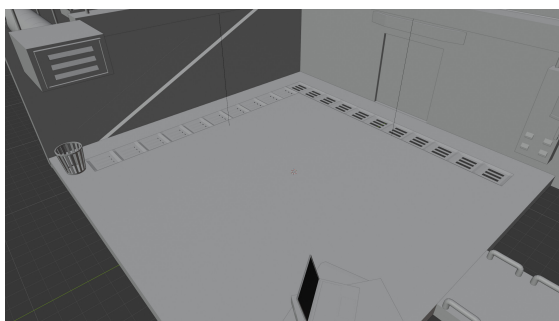


Figura 3.8: Vista de *Viewport* do piso do cenário do jogo.

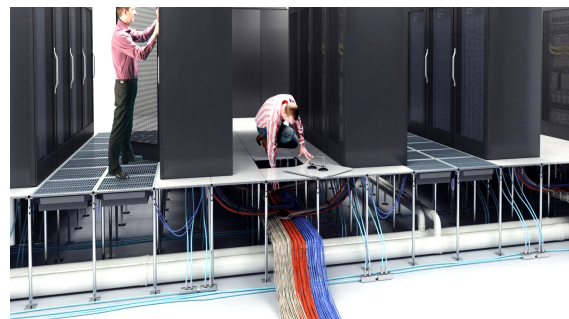


Figura 3.9: Ilustração do pavimento técnico.

Analisando o cenário da Figura ?? é possível observar a presença de umas caixas no chão, que apresenta algumas perfurações, estas caixas têm como finalidade representar o pavimento técnico ou pavimento elevado, representado na Figura ??, que é um elemento comum em data centers, e tem a função de permitir a passagem de cabos e sistemas de refrigeração, além de facilitar o acesso aos equipamentos instalados no chão do data center. A representação no jogo é uma adaptação do pavimento técnico real, onde as caixas foram simplificadas, e a fiação de cabos foi desconsiderada. A sua localização no cenário, também serve de marcação para o jogador, uma vez que sempre que existe a compra de um novo servidor, o mesmo é colocado atrás de uma dessas caixas.



Figura 3.10: Vista de *Viewport* do switch de rede.



Figura 3.11: Ilustração do switch de rede.

Além da estrutura do próprio cenário, os elementos de decoração presentes nele, são igualmente relevantes para a caracterização do espaço. Assim sendo houve o cuidado de criar elementos que remetem para o contexto de data center, tais como o switch representando na Figura ?. O switch é um equipamento de rede utilizado para conectar vários dispositivos em uma mesma rede local. A representação do switch no jogo é uma adaptação de um switch real, mostrado na figura ?.

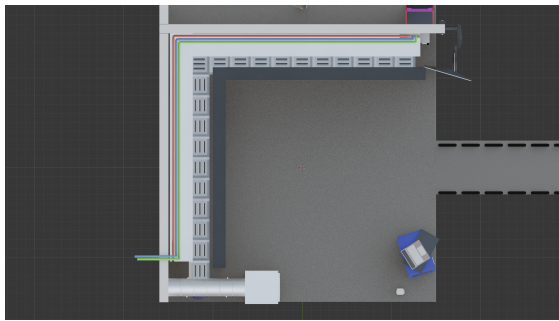


Figura 3.12: Vista de *Viewport* do caminho de cabos (Vista de cima).

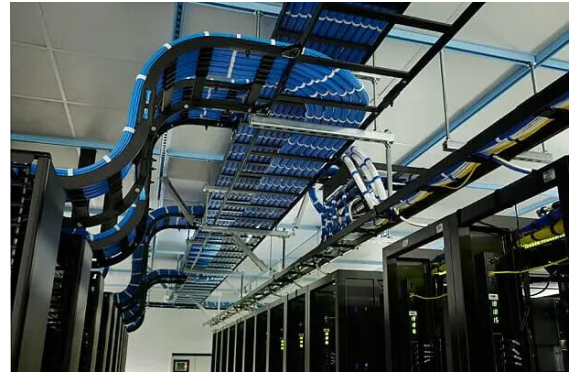


Figura 3.13: Ilustração do caminho dos cabos.

Na parte superior do cenário, existe outro detalhe de decoração, bastante importante para a caracterização do espaço, que é o caminho de cabos, representado na Figura ?? . Um caminho de cabos, é na sua essência, uma calha colocada geralmente no teto, que tem a função de organizar, proteger e transportar os cabos de rede, energia e outros pelo espaço do data center. Na Figura ?? pode ser vista uma representação realista do mesmo.

Na Figura ?? é possível observar uma representação de um sistema de ventilação e refrigeração, que é um elemento essencial em data centers, uma vez que tem a função de manter a temperatura e a umidade do ambiente controladas, garantindo o bom funcionamento dos equipamentos. A representação do sistema de ventilação e refrigeração no jogo é uma adaptação de um sistema real, onde os detalhes foram simplificados para se adequar ao estilo visual do jogo.

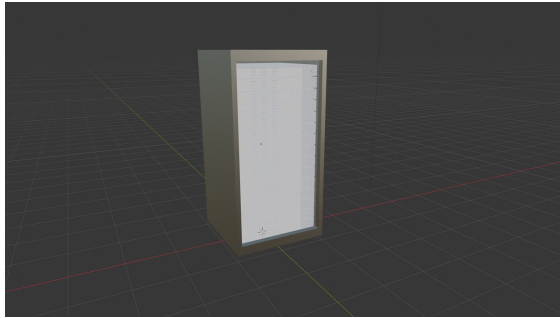


Figura 3.14: Vista de *Viewport* do servidor.



Figura 3.15: Representação do servidor da Amazon.

A representação de servidores no jogo, assim como se pode ver na Figura ??, é uma adaptação de *racks* de servidores reais, conforme é mostrado na Figura ?. Os *racks* de servidores são estruturas metálicas que abrigam os equipamentos de um data center, como servidores, switches, roteadores, etc... A representação de servidores no jogo, foi simplificada, no sentido de homogeneizar a aparência de todos os equipamentos e função os limitando apenas servidores, no sentido de computadores.

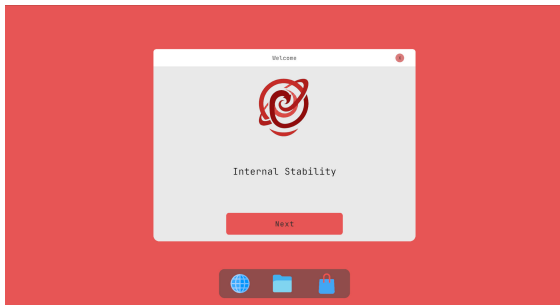


Figura 3.16: *UI* da instalação e configuração de um sistema operativo.

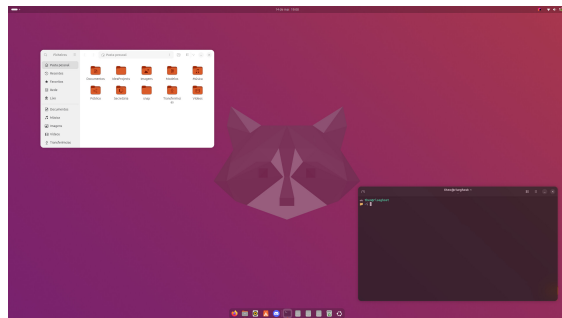


Figura 3.17: Ambiente gráfico Gnome.

Por fim, além de elementos gráficos de cenário, elementos de *UI*, tais como a interface de configuração de um sistema operativo, foram igualmente inspirados em referência reais. Na Figura ?? é mostrado a interface gráfica Gnome, que é um ambiente gráfico vastamente utilizado em sistemas operativos Linux, embora a sua utilização em data centers não seja tão comum, ainda assim, é bastante dissolvida no ambiente de tecnologia. Devia a ser uma interface gráfica mais amigável e intuitiva,

servi como inspiração para a construção da interface vista na Figura ??.

3.3 Desenvolvimento do conceito de jogo

Este projeto consiste na construção de um vídeo jogo, onde o jogador é responsável por fazer a gestão de um pequeno data center, onde o objetivo é ganhar dinheiro.

No entanto, durante todo o percurso de *gameplay*, o jogador terá várias etapas a cumprir, como, comprar equipamento, configurar o equipamento, resolução de pequenos puzzles disponibilizados sobre a forma de *mini games*, gestão de recursos económicos e combate contra ameaças, como, por exemplo, vírus e ataques de *hackers*.

Nas subseções seguintes, será desenvolvido de forma mais detalhada cada um dos pontos principais do conceito do jogo.

3.3.1 Compra e configuração de equipamento

Durante a compra de equipamentos para o data center, o jogador tem a sua disposição uma interface completa de configuração, onde é possível escolher componentes como:

- Processadores;
- RAM;
- Discos de vários tipos;
- Placas de rede;
- Sistema Operativo;
- Sistemas de refrigeração (Está presente de forma simplificada).

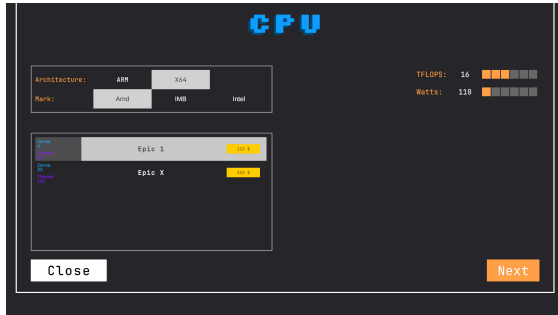


Figura 3.18: Captura de tela do jogo – Configuração de CPU.



Figura 3.19: Captura de tela do jogo – Configuração de discos.

Na Figura ?? e na Figura ?? é possível observar a interface de configuração de CPU e de discos, respetivamente, onde o jogador tem a possibilidade de escolher entre vários tipos de componentes, cada um com as suas próprias características e preços, o que torna a escolha do equipamento uma parte importante da estratégia do jogo. Além disso, cada componente tem um impacto direto no consumo de energia (*Watts*) assim como no desempenho daquele servidor (*TFLOPS*). As unidades utilizadas (*Watts* e *TFLOPS*) são unidades reais, e o uso server para reforma a imersão do jogador, além de tornar o jogo mais realista, e permitir a adição de conteúdo educativo de forma lúdica.

3.3.2 Sistema de economia

O jogo integra um sistema de economia dinâmico que exige ao utilizador a gestão estratégica de recursos financeiros. Esta gestão abrange não só o investimento em bens de capital — como a aquisição de equipamentos e respetivos *upgrades* — mas também a administração de despesas operacionais correntes, nomeadamente os custos associados ao consumo de eletricidade, sistemas de refrigeração, abastecimento de água e serviços de conectividade à internet.

A componente económica é processada em tempo real, mantendo a continuidade do fluxo financeiro independentemente do estado da sessão do jogador. Este mecanismo assegura que:

- **Débitos Operacionais:** As despesas fixas e variáveis são calculadas com base em períodos predefinidos (x minutos). Caso o jogador se ausente do jogo por um

período $n \cdot x$ (onde $n \geq 1$), o montante debitado será igual a n , da próxima vez que abrir o jogo.

- **Fluxos de Rendimento:** De igual modo, a geração de receita é contínua para serviços de subscrição ou fidelização. Um exemplo prático é o aluguer de infraestrutura de *data center* para alojamento de *websites*, onde os ganhos são contabilizados proporcionalmente ao tempo decorrido, garantindo a persistência económica do ecossistema.

Este sistema está presente na versão apresentada do projeto, embora com uma redução no número de variáveis a gerir, tais como a ausência de custos relacionados com sistema de refrigeração, abastecimento de água e serviços de internet. Sendo assim as únicas despesas presentes sendo:

- Custos relacionados com o consumo de eletricidade;
- Custos relacionados com a compra de equipamentos.

Todavia, creio que o sistema de economia presente cumpre a função de transmitir a ideia real de gestão de recursos, além de proporcionar um gancho de interesse para o jogador voltar a entrar no jogo, com o intuito de receber os pagamentos pendentes.

3.3.3 Sistema de combate

O conceito do jogo conta com uma componente de combate, onde o jogador teria a sua disposição um sistema de combate e *buids* semelhantes a jogos de **RPG!** (**RPG!**), mas adaptado ao contexto do jogo. Ou seja, os atributos de ataque, defesa, velocidade, etc, seriam substituídos por atributos relacionados com o contexto do jogo, como, por exemplo, poder de processamento, largura de banda, algoritmos de segurança, etc...

A intenção desta componente de combate, seria quebrar um pouco do ciclo de jogo, e proporcionar ao jogador uma experiência mais dinâmica e variada, utilizando da temática para construir uma experiência diferente e divertida, onde o jogador poderia ser introduzido a conceitos e expressões técnicas de uma forma lúdica sem perder o foco na diversão.

Segundo Zhang (2009), os jogos não comunicam apenas por imagens ou palavras, mas por meio de processos. Ou seja, quando utiliza de um contexto de Data Center

adaptado a um **RPG!**, estamos a criar um modelo de como esse sistema funciona. Segundo o autor, essa "aprendizagem por simulação" gera uma curiosidade intrínseca, pois o jogador tenta decifrar a lógica do sistema real através das regras do jogo.

Embora a ideia apresentada a cima seria a ideal para o conceito do jogo, infelizmente devido a sua complexidade para o seu desenvolvimento, o projeto apresentado teve de ser adaptado, no sentido em que vês de ter o sistema de **RPG!**, o jogo terá uma metralhadora no cenário em que o jogador terá de a usar para destruir os inimigos (vírus e *hackers*), representados sobre a forma de *drones*, que iram aparecer durante momentos da *gameplay*. Embora, essa adaptação carece em termos de complexidade e profundidade de contexto, limitando a precessão do jogador comparado com o sistema de **RPG!**, creio mesmo assim ser relevante uma vez que introduz um elemento de combate, e quebra uma pouco do ciclo de jogo, que se poderia tornar repetitivo, além de proporcionar uma experiência mais dinâmica e divertida para o jogador.

3.3.4 Estilo e estética

O jogo é caracterizado por ser um jogo 3D com uma perspetiva isométrica, onde o jogador tem uma visão geral do cenário, e pode interagir com os diferentes elementos do jogo para cumprir os objetivos propostos.

O jogo faz uso de elementos de fantasia e ficção, principalmente na estética, visando criar um ambiente mais divertido e envolvente para o jogador.

3.4 *Gameplay Loops*

Um *gameplay loop* é ciclo de ações que se repete, sendo tipicamente composto por 3 etapas:

- Objetivo;
- Desafio;
- Recompensa;

que definem a experiência do jogo. Estes ciclos estão presentes em várias partes do projeto desenvolvido, tais como na compra de equipamentos, resolução de tarefas(pedidos de clientes) e combate.

Posto isto, nas subsecções seguinte serão apresentados o *core gameplay loop*, assim como os *gameplays* secundário.

3.4.1 *Gameplay loop*

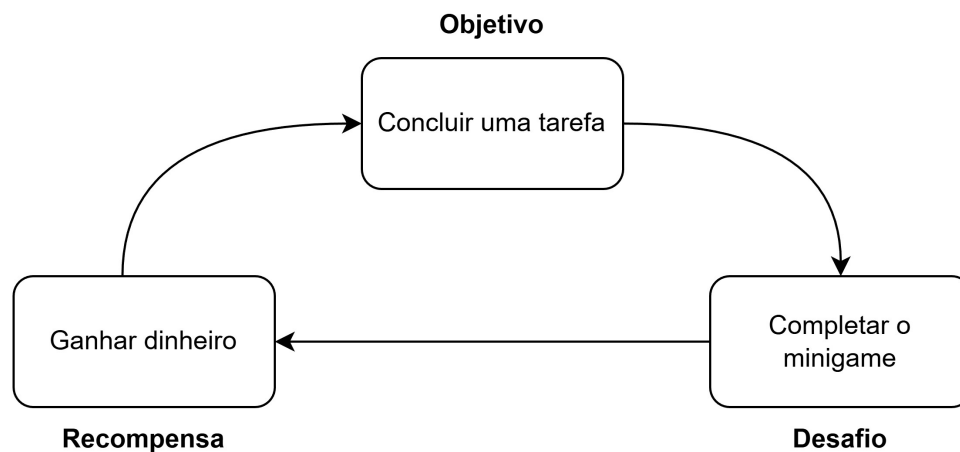


Figura 3.20: Diagrama do *core gameplay loop* do jogo desenvolvido.

A Figura ?? apresenta o diagrama do *core gameplay loop*, ou seja, o ciclo de ações principais do jogo. A tarefa referida no diagrama, diz respeito aos pedidos dos clientes. Esses pedidos aparecem de forma constante e periódica durante toda a *gameplay* além de representarem a principal forma do jogador conseguir dinheiro.

A seguir apresenta-se os *gameplay loops* secundários, ou seja, os ciclos de ações relacionados com a compra e configuração de equipamentos, e o ciclo de ações relacionado com o combate.

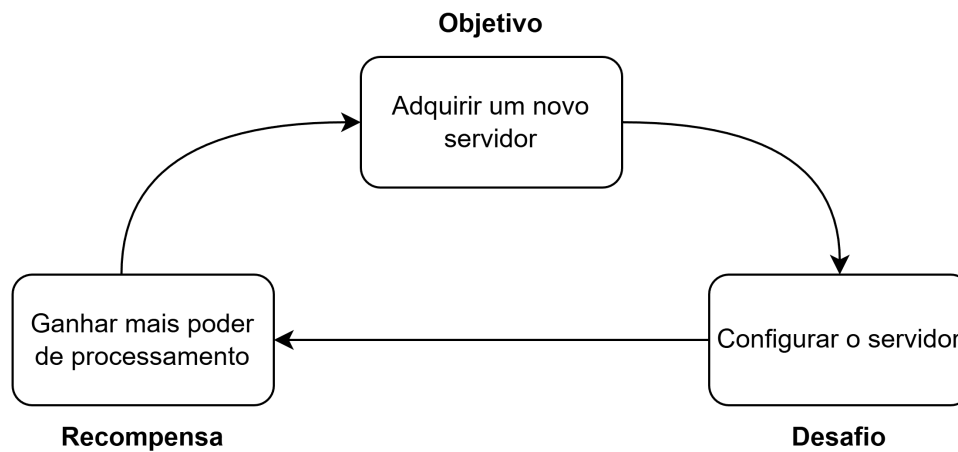


Figura 3.21: Diagrama do *gameplay loop* referente à configuração de servidores.

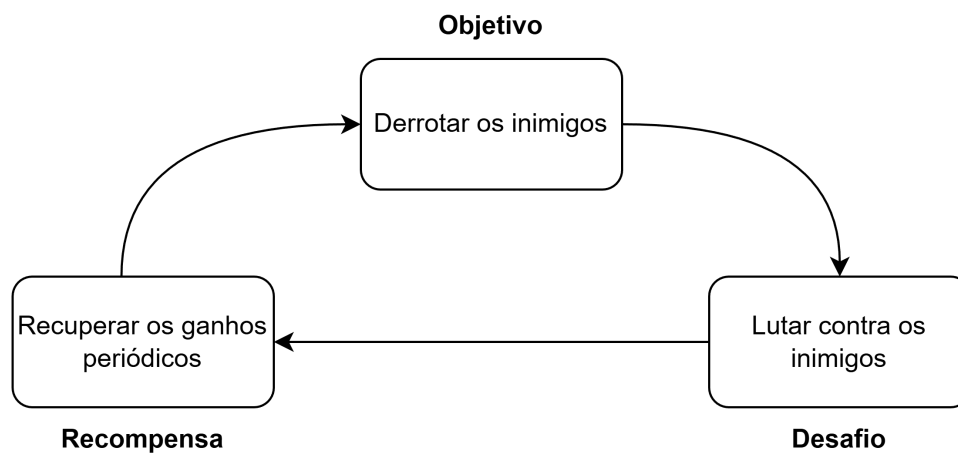


Figura 3.22: Diagrama do *gameplay loop* referente ao combate.

Capítulo 4

Implementação do Jogo

4.1 Ferramentas e Tecnologias Utilizadas

Várias ferramentas e diferentes tecnologias, foram utilizadas durante todo o processo de desenvolvimento deste projeto, permitindo assim o seu desenvolvimento e facilitando a sua implementação. Considerando ferramentas de construção de conteúdo digital até linguagens de programação. Posto isto, as ferramentas e tecnologias utilizadas foram:

- **Blender:** *Software* de modelação 3D. Este *software* foi utilizado para a modelação completa do cenário, assim como para a realização de ajuste no personagem (Criação de textura);
- **Unity:** *Game Engine*, *software* principal para o desenvolvimento do projeto. Ele é responsável pela junção dos *asset's* com o código. Permitindo assim a construção de uma experiência interativa. Além de ser utilizado para a criação de *shaders*, que influenciam na construção visual do projeto;
- **Chat 3D:** Ferramenta de **IA! (IA!)**. Utiliza para desenvolver o modelo 3D do personagem do jogo utilizando imagens 2D como base para a sua criação;
- **Gemini (Nano banana):** Ferramenta de **IA!**, utilizada para desenvolver o conceito do personagem, assim como utilizada para a criação de imagens ilustrativas do mesmo, que foram utilizadas no Chat 3D;
- **Processing:** *Framework* de Java, que permite a criação de imagens 2D. Esta

tecnologia foi utilizada para a criação de algumas das partículas 2D presentes na UI!;

- **Gimp e Affinity:** Ferramentas de edição de imagem. Foram utilizadas principalmente para ajustar as imagens geradas pelo Gemini;
- **C#:** Linguagem de programação principal. É utilizada para a construção completa do jogo dentro da Unity. Responsável pela lógica do jogo, assim como implementação de sistema de *gameplay*;
- **Java:** Linguagem de programação utilizada no *framework* Processing. Utilizada para a construção de certos *assets* 2D;

4.2 Engenharia de Software

4.2.1 Diagrama de sistema de jogo

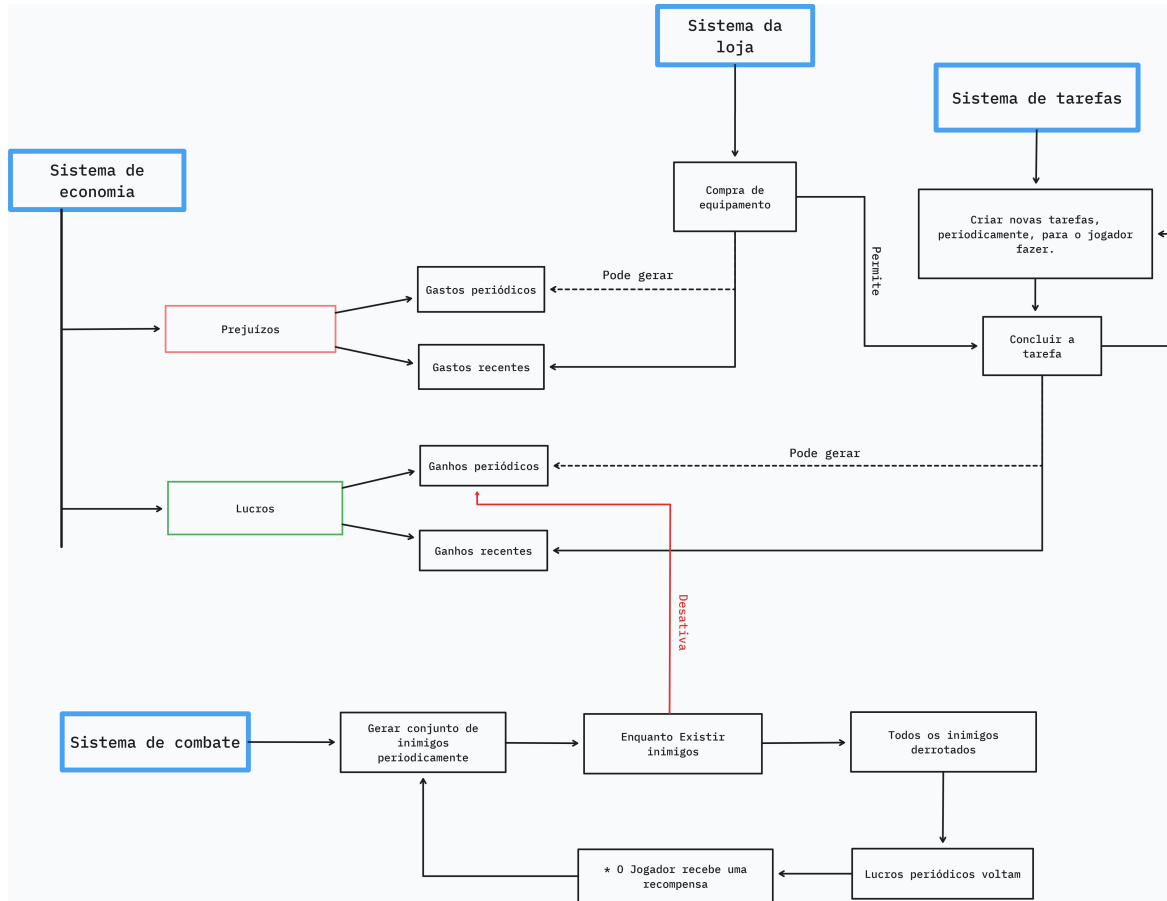


Figura 4.1: Diagrama de sistema do jogo.

A Figura ?? apresenta o diagrama de sistema do jogo, onde é possível observar os vários sistemas que compõem o jogo, assim como a interação entre eles. Como é observável, todos os sistemas interagem com o sistema de economia, uma vez que esse sistema está fortemente ligado com a condição de derrota. Além disso, é observável que o único sistema que permite com que o jogador ganhe dinheiro é o sistema de tarefas, o que faz com que seja através dele que o jogador consiga progredir no jogo, e como tal é o sistema que o jogador interage mais durante o jogo.

No diagrama, é possível observar que no sistema de combate, existe uma recompensa para o jogador. Embora essa recompensa faça parte da idealização do sistema de combate, ela não foi implementada, devido à complexidade.

4.2.2 Diagramas de caso de uso

Diagramas de caso de uso, descrevem a funcionalidade de um sistema através da interação entre atores (utilizadores/sistema). No contexto deste projeto os utilizadores são os jogadores e o sistema são os vários sistemas que compõem o jogo. Nas subsecções subsequentes serão apresentados os diagramas de caso de uso dos três principais sistemas: Tarefas, Loja e combate.

4.2.2.1 Tarefas

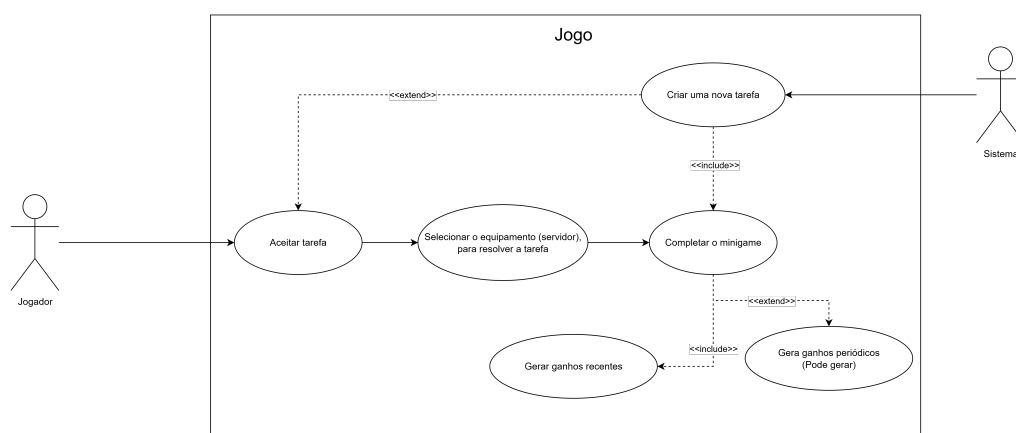


Figura 4.2: Diagrama de Caso de uso - Tarefas

O diagrama da Figura ??, retrata a interação entre o jogador e o sistema de tarefas que periodicamente cria tarefas para o jogador fazer, as quais, o jogador pode aceitar ou recusar. Uma vez que, cada tarefa exige recursos, o jogador pode escolher recusar uma vez que pode não os ter ou acreditar que não compensa.

Além disso, assim como mostra o diagrama, ao concluir cada tarefa o jogador recebe uma certa quantidade de dinheiro, podendo em certos casos receber outra quantidade de dinheiro de forma periódica.

4.2.2.2 Loja

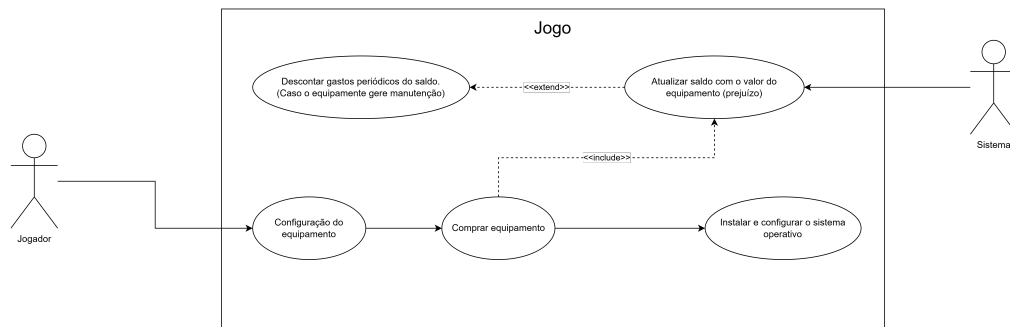


Figura 4.3: Diagrama de Caso de uso - Loja

A Figura ?? ilustra como o jogador interage com o sistema de Loja. Os pontos mais relevantes das funcionalidades apresentadas são:

- **Configuração de equipamento:** Uma vez que nesta etapa o jogador pode montar o servidor com o equipamento que quiser, dando assim liberdade para que o jogador possa ser criativo e conseguia balancear aspetos relevantes (como performance, armazenamento, refrigeração) face ao custo envolvido.
- **Instalação de sistema operativo:** É um passo à posterior a configuração do equipamento, onde o jogador tem que faz uma simulação de instalação de um sistema operativo, onde pode escolher o nome daquele servidor assim como a *password*. Este passo é importante para a construção da experiência, uma vez que é um processo que acontece na realidade, e que tem um certo grau de complexidade, o que pode ajudar a construir uma experiência mais imersiva. Além disso, o nome atribuído ao servidor, é utilizado posteriormente para o jogador conseguir identificar o seu servidor, uma vez que o jogador pode ter mais do que um servidor.

4.2.2.3 Combate

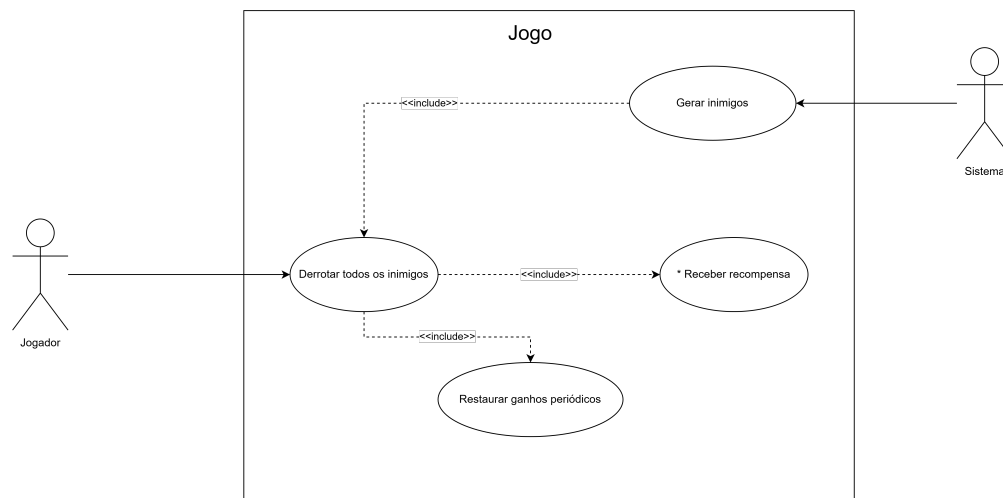


Figura 4.4: Diagrama de Caso de uso - Combate

A Figura ??, ilustra a interação entre o jogador e o sistema de combate. Neste sistema, a funcionalidade mais relevante é a "Restaurar ganhos periódicos", uma vez que a partir do momento que o sistema combate "Gera inimigos" os ganhos periódicos são desativados, no entanto, os custos (prejuízo) não são desativados, o que deixa o jogador numa situação de risco. O jogador tem de agir rápido para conseguir restaurar os ganhos periódicos, caso contrário, pode perder uma grande quantidade de dinheiro, o que pode levar a uma situação de falência, impossibilitando o pagamento dos custos periódicos, o que leva o jogador a perder o jogo.

4.3 Implementação

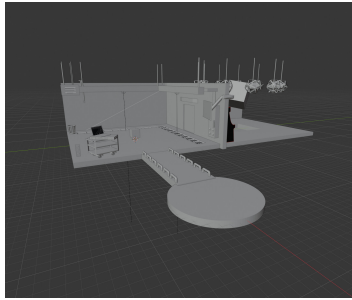
A implementação constitui a fase em que os planos idealizados são concretizados. Nesse sentido, a descrição dos processos será detalhada ao longo desta secção, na qual cada subsecção abordará a implementação de uma componente específica do projeto.

4.3.1 Modelos 3D - Blender

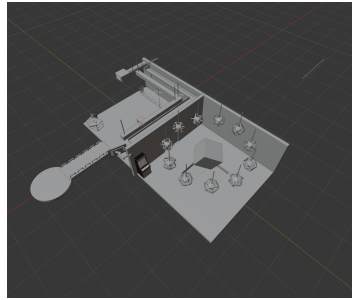
Dada a natureza do projeto, os modelos 3D assumem um papel de destaque, visto que é através destes que o jogador experiencia o mundo e realiza interações. Como tal, o seu desenvolvimento deve ser estruturado para maximizar a experiência de *gameplay*, mantendo um padrão visual reconhecível e coerente.

Neste sentido, a maioria dos modelos 3D foi desenvolvida por autoria própria, recorrendo ao *software* Blender.

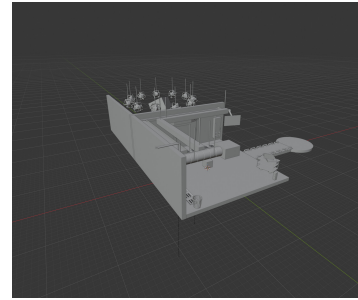
4.3.1.1 Cenário



(a) Cenário Vista de *ViewPort* 1



(b) Cenário Vista de *ViewPort* 2



(c) Cenário Vista de *ViewPort* 3

Figura 4.5: Vistas de *ViewPort* do cenário.

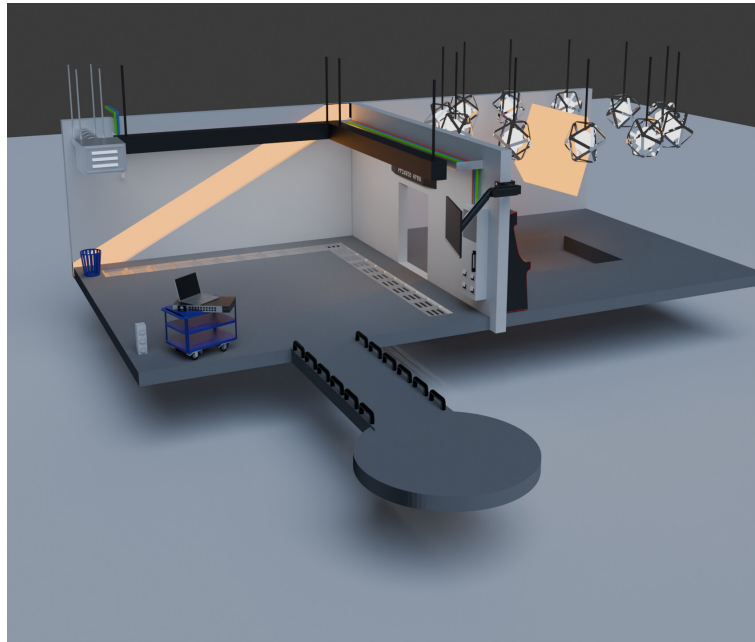


Figura 4.6: Vista renderizada do cenário do jogo.

As Figuras ?? e ?? ilustra o trabalho desenvolvido para o cenário. Todos os subcomponentes dos cenários, tais como, mesas, computadores, e outros detalhes desenvolvidos com a ajuda do *software*.

4.3.1.2 Canhão

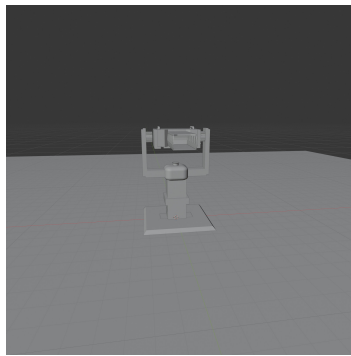
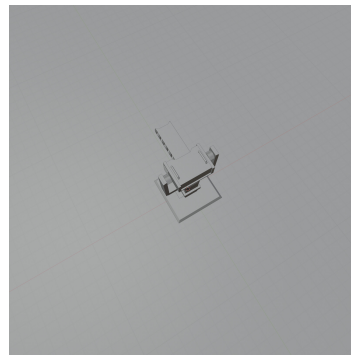
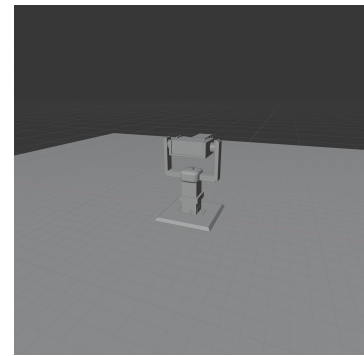
(a) Cenário Vista de ViewPort
1(b) Cenário Vista de ViewPort
2(c) Cenário Vista de ViewPort
3

Figura 4.7: Vistas de ViewPort do canhão.

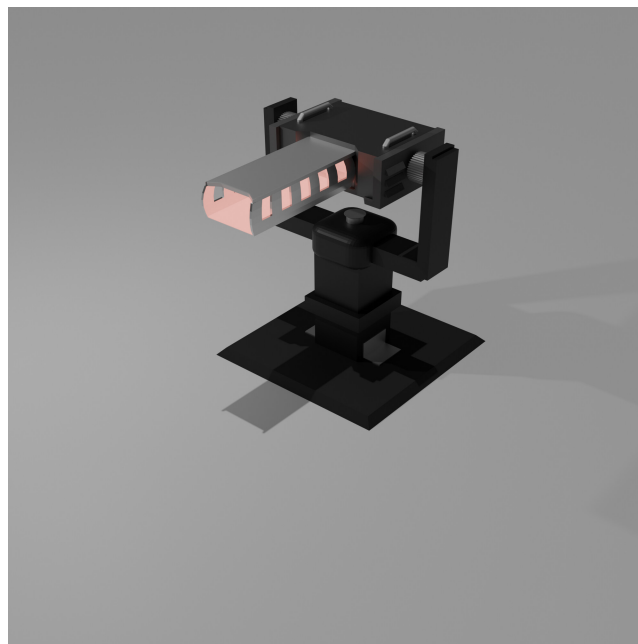


Figura 4.8: Vista renderizada do canhão.

As Figuras ?? e ?? ilustram o modelo do canhão. Os processos de modelação assim como de pintura de materiais, foram desenvolvidos dentro do *software* Blender.

Com o design deste modelo, pretenda-se transmitir uma sensação de futurismo moderada, mas que, ao mesmo tempo, harmozine com o cenário restante.

4.3.2 Imagens e *assets* 2D

Embora não sejam tão utilizados quanto os ativos 3D, os ativos 2D apresentam um papel bastante importante, sobretudo para elementos de **UI** e para a idealização do contexto de elementos.

4.3.2.1 Idealização do personagem

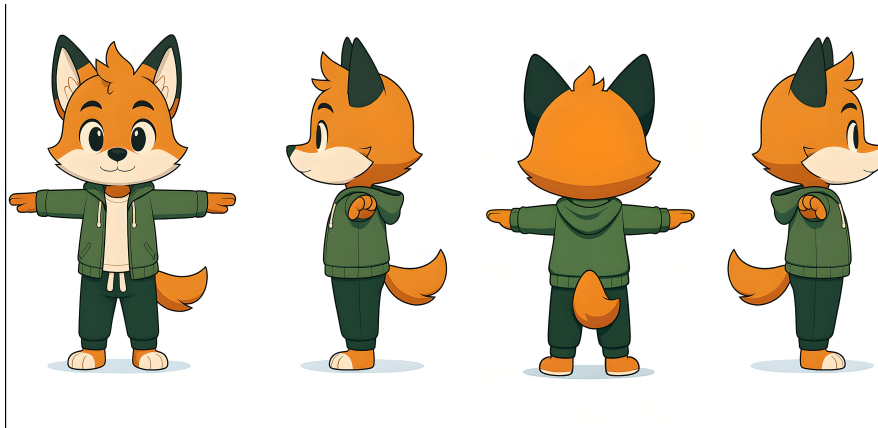


Figura 4.9: Ilustração do personagem feita com o uso da IA! – Nano Banana (Gemini)

Durante o processo criativo do personagem, utilizou-se a ferramenta Nano Banana do Gemini para gerar a Figura ???. O objetivo foi representar visualmente as suas características físicas, de maneira que, esta possa ser utilizada por outras ferramentas de IA, tais como o Chat 3D, para a criação de um modelo tridimensional.

4.3.2.2 *Frames* para a criação de animações 2D

No contexto de jogos de vídeo, transmitir *feedbacks* ao jogador é um dos processos mais importantes, uma vez que eles funcionam como uma confirmação que uma ação foi realizada e/ou ajudam a construir a experiência facilitando a evocação de emoções para jogador.

Feedbacks podem ser transmitidos, através de **VFX!** (**VFX!**), animações e efeitos sonoros, por exemplo, efeitos visuais de faíscas no disparo de uma arma, até animações do personagem a andar, etc...

Posto isto, para a construção de certos *feedbacks* visuais, foram criados *frames* 2D, utilizando o *framework* Processing, os quais foram posteriormente utilizados para a

construção de animações 2D dentro da Unity.

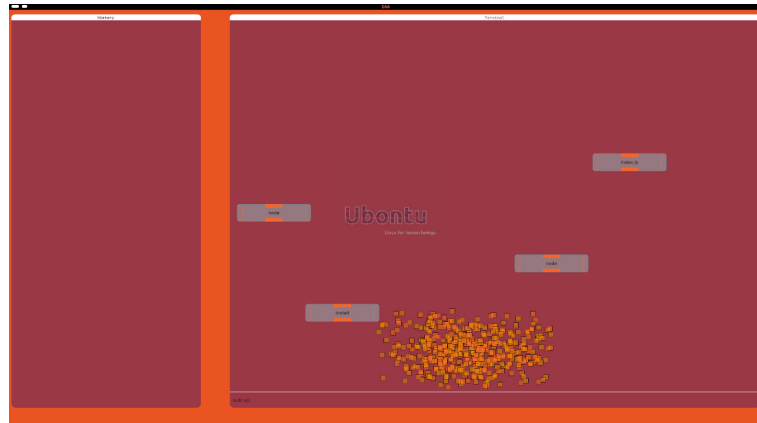


Figura 4.10: VFX! criado com o uso de *frames* 2D criados com o *framework* Processing.

A Figura ?? representa um caso prático onde os *frames* criados com o *framework* Processing foram utilizados para a construção de um efeito visual, quando o jogador acerta uma palavra no mini-jogo.

Além desse caso, na Figura ?? é possível observar outro caso da utilização do Processing, onde foi utilizado para criar um conjunto de imagens 2D, as quais foram utilizadas para criar uma animação do menu principal do jogo, a qual é reproduzida quando o jogador inicia o jogo.



Figura 4.11: Animação do menu principal criada *frame-by-frame* com a ajuda do *framework* Processing.

4.3.3 Personagem

4.3.3.1 Modelo 3D

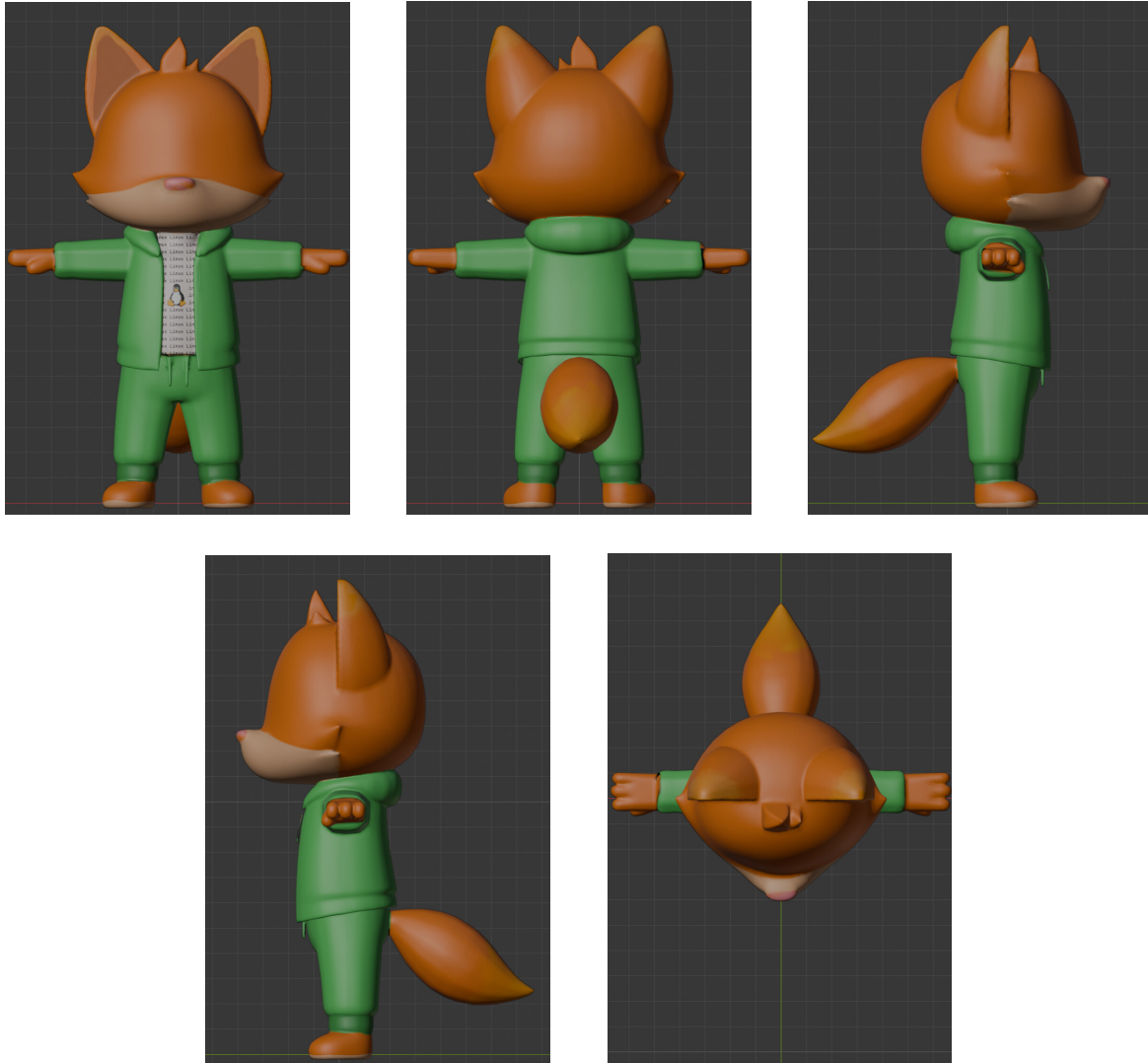


Figura 4.12: Modelo 3D do personagem, em várias vistas.

O modelo 3D (*mesh*) do personagem (Figura ??) foi criado com o auxílio da ferramenta de **IA!**, Chat 3D, usando como referência a Figura ??. No entanto, a pintura de materiais e a construção de texturas, foram realizadas no *software* Blender, utilizando o *software* GIMP para desenvolver a textura da camiseta branca. O visual do personagem foi inspirado na mascote do **Mozilla Firefox**, uma vez que o jogo tem como tema central a internet, e o Firefox é um dos símbolos mais reconhecíveis da internet, o que faz com que seja uma fonte de inspiração bastante relevante para o projeto.

4.3.3.2 Esqueleto e animações

Um esqueleto de um modelo 3D é uma estrutura interna composta por um conjunto de “ossos” virtuais organizada de forma hierárquica, que serve para controlar a deformação e o movimento de um objeto tridimensional.

O esqueleto do modelo 3D é relevante, pois é através dele que é possível criar animações para o modelo, tais, como animações de caminhada, corrida, *idel*, etc. . . . O esqueleto do personagem foi criado no *website Mixamo*, onde também foram retidas as animações de caminhada, corrida e *idle*.

Após isso, o modelo 3D do personagem, com o esqueleto e as animações, foram importados para a Unity, onde foram feitos ajustes para que as animações pudessem ser reproduzidas de forma fluida e natural.

Capítulo 5

Testes e Avaliação do Jogo

5.1 ...

...

Capítulo 6

Conclusões e Trabalho Futuro

6.1 Contribuições

...

6.2 Trabalho Futuro

...

Bibliografia

- Adams, E. (2014). *Fundamentals of game design*. Pearson education.
- Chen, J. (2007). Flow in games (and everything else). *Communications of the ACM*, 50(4), 31–34.
- Claudiu Kiss. (2019, janeiro). *PC Building Simulator*. The Irregular Corporation. https://store.steampowered.com/app/621060/PC_Building_Simulator/
- Coredumping. (2015, maio). *Software Inc*. https://store.steampowered.com/app/362620/Software_Inc/
- Eyal, N. (2014). *Hooked: How to build habit-forming products*. Penguin Uk.
- Hosseini, S., & Vakili, R. (2018). Game theory approach for detecting vulnerable data centers in cloud computing network. <https://onlinelibrary.wiley.com/doi/abs/10.1002/dac.3938>
- Hovgaard Games. (2020, abril). *Startup Company*. https://store.steampowered.com/app/606800/Startup_Company/
- Hunicke, R., LeBlanc, M., Zubek, R., et al. (2004). MDA: A formal approach to game design and game research. 4(1), 1722.
- Loading Home. (2017, dezembro). *Grey Hack*. https://store.steampowered.com/app/605230/Grey_Hack/?l=portuguese&60mrXkXAee6f8R=ecjb6O
- Shay, L., & Langbeheim, E. (2022). Scaffolding math learning and motivation by integrating casual play in computer-based practice for fifth graders. *Proceedings of the 16th International Conference of the Learning Sciences-ICLS 2022*, pp. 1934-1935.
- Silva, F. G. M. (2019). Practical Methodology for the Design of Educational Serious Games. *Information*. <https://www.mdpi.com/2078-2489/11/1/14>
- Waseku. (2026). *Data Center Demo* (Versão Demo). Steam. https://store.steampowered.com/app/4376050/Data_Center_Demo/

- Yu, W. B., MacCormick, D., Zaman, L., & Mirza-Babaei, P. (2019). Getting the Player's Attention: Comparing the Effectiveness of Common Notification Types in Task Management Games. *Extended Abstracts of the Annual Symposium on Computer-Human Interaction in Play Companion Extended Abstracts*, 797–804.
- Zhang, Y. (2009). Ian Bogost, *Persuasive Games: The Expressive Power of Videogames*.